Github Link: https://github.com/David-Chan-Ho2/MDM-Advanced-RAG

Over the past three weeks, our team built an Advanced RAG pipeline that automates technical attribute extraction from approximately 5,000 product documents. Week 1 was focused on establishing the foundational architecture. We created a document ingestion pipeline capable of handling PDF, DOCX, XLSX, HTML, and CSV files. Early in the process, we decided to use local sentence-transformers instead of OpenAI embeddings. This choice helped reduce costs and avoid external dependencies. We selected Qdrant as our vector database because it provided better filtering capabilities and hybrid search functionality. For chunking, we implemented 800-token chunks with 100-token overlap. This ensured that table structures remained intact, which was critical for preserving technical specifications.

In Week 2, the architecture became significantly more sophisticated. We implemented hybrid retrieval by combining dense vector search with BM25 sparse search. We fused the results using reciprocal rank fusion and cross-encoder re-ranking. This approach allowed us to capture both semantic meaning and exact keyword matches. It helped the system locate part numbers and voltage specifications buried in lengthy documents. We also introduced query decomposition and designed a five-agent extraction framework. Each agent specialized in extracting specific data types including product identifiers, dimensions, electrical properties, materials and compliance, and performance metrics. An orchestrator managed all agents and combined their outputs. It validated confidence scores to ensure quality. The architecture proved both performant and maintainable.

Week 3 revealed a critical architectural issue. Our retrieval system was filtering by document_id when it should filter by product_id. Although this distinction might seem minor, it had significant implications for correctness. We refactored the entire extraction interface and renamed core functions. We updated metadata filters consistently throughout the codebase. This change enabled the pipeline to handle products whose data spanned multiple files. We also integrated output serialization directly into the batch runner. This created a complete end-to-end pipeline that generated JSON exports compatible with PIM systems. After testing everything, we confirmed that all components functioned correctly.

This capstone project was the most comprehensive and encompassing application experience we have had throughout our computer science program. We applied database concepts like vector indexing and metadata filtering to solve a real business problem. The multi-agent design pattern showed us how to break down complex challenges into modular, testable components. We drew on foundational knowledge of natural language processing and machine learning. We evaluated tradeoffs between different retrieval approaches. We strengthened our software engineering skills by writing testable code and managing dependencies. We also learned to refactor code for correctness and analyze data flow across

systems. Most importantly, this project filled critical gaps in our practical experience with AI implementation. We learned how to integrate multiple AI techniques into a cohesive system and make real tradeoffs between competing approaches. The most important lesson was recognizing that production systems require balancing multiple concerns like cost, accuracy, maintainability, and scalability. We learned to make deliberate architectural decisions early on. We learned to identify when assumptions are wrong and refactor confidently when needed. Seeing the entire development process from initial specification through final implementation gave us invaluable insight into software engineering. This project wrapped together everything we learned and showed us how theory connects to practice. The experience of transforming a specification into a working implementation has helped us bridge the gap between the classroom and building systems that people actually use.